Assignment - 6

* Problem Statement.

Write a program to implement polynomial operations using a circular linked list. The program should support the following operations:

(1) Create polynomial by inserting terms.
(2) Display the polynomial in a readable mathematical form.
(3) Add two polynomials display the resultant polynomial.

* Objectives

(1) To implement a polynomial ~~a~~ representation using a circular ~~queue~~ linked list by dynamically creating & storing polynomial terms.

(2) To perform fundamental polynomial operations such as creating, displaying & adding the polynomial using circular linked list.

(3) To understand advantage of circular list linked in managing dynamic data structures & efficiently handling polynomial arithmetic by combining like terms.

* Pseudo code
write pseudo code for create polynomial display polynomial, add polynomial using circular linked list.

(1) Create polynomial
(i) Enter coefficient & exponent.
(ii) Create nodes and insert terms.
(iii) Link the last node to the first node.

(2) Display Polynomial
(i) Set temp = head
(ii) Display each term.
(iii) Stop when temp reaches head again.

(3) Add polynomial
(i) Compare exponent of both polynomials.
(ii) Add coefficient if exponents are equal
(iii) Copy terms with different exponents.
(iv) Display the resultant polynomial.

* Time Complexity

	Operation	Time Complexity
1.	Create polynomial	O(n)
2.	Display polynomial	O(n)
3.	Add two Polynomial	O(n+m)

* Conclusion

The program is successfully implements polynomial creation, display & addition using circular linked list, demonstrating efficient dynamic memory management & polynomial manipulation.

* FAQ's

1. Why is circular linked list used to represent polynomials instead of linear linked list?

→ It allows continuous traversal of polynomial terms because the last node points back to the first node.

2. What information is stored in each node of the circular linked list?

→ Each node stores the coefficient, exponent, and pointer to the next node.

3. What happens if one of the polynomial is empty during addition.

→ The result is the other polynomial because adding zero does not change the polynomial.

PRACTICAL 6

POLYNOMIAL USING CIRCULAR LINKED LIST

CSF20030 – Data Structures Lab

Experiment 06: Circular Linked List Operations

Aim: Create, display, and add two polynomials using a Circular Linked List.

C Program

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int coeff;
    int exp;
    struct Node *next;
} Node;
typedef struct {
    Node *tail;
} Polynomial;
/* Create an empty circular polynomial */
void initPolynomial(Polynomial *p) {
    p->tail = NULL;
}
/* Insert a term in descending exponent order.
   If the exponent already exists, combine coefficients. */
void insertTerm(Polynomial *p, int coeff, int exp) {
    if (coeff == 0) return;
    Node *cur, *prev, *node;
    if (p->tail == NULL) {
        node = (Node *)malloc(sizeof(Node));
        if (node == NULL) {
            printf("Memory allocation failed.\\n");
            exit(EXIT_FAILURE);
        }
        node->coeff = coeff;
        node->exp = exp;
        node->next = node;
        p->tail = node;
        return;
    }
    Node *head = p->tail->next;
    cur = head;
    prev = p->tail;
    do {
        if (cur->exp == exp) {
            cur->coeff += coeff;
            if (cur->coeff == 0) {
                if (cur == cur->next) {
                    free(cur);
                    p->tail = NULL;
                } else {
                    prev->next = cur->next;
                    if (cur == p->tail) p->tail = prev;
                    free(cur);
                }
            }
            return;
        }
        if (cur->exp < exp) break;
        prev = cur;
        cur = cur->next;
    } while (cur != head);
    node = (Node *)malloc(sizeof(Node));
    if (node == NULL) {
        printf("Memory allocation failed.\\n");
        exit(EXIT_FAILURE);
    }
    node->coeff = coeff;
    node->exp = exp;
    if (cur == head && cur->exp < exp) {
        node->next = head;
        p->tail->next = node;
    } else if (cur == head) {
        node->next = head;
        p->tail->next = node;
        p->tail = node;
    } else {
        node->next = cur;
        prev->next = node;
    }
}
/* Read a polynomial */
void createPolynomial(Polynomial *p) {
    int n, coeff, exp;
    printf("Enter number of terms: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Invalid number of terms.\\n");
        exit(EXIT_FAILURE);
    }
    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent for term %d: ", i + 1);
```

```
        if (scanf("%d%d", &coeff, &exp) != 2 || exp < 0) {
            printf("Invalid term. Exponent must be non-negative.\\n");
            exit(EXIT_FAILURE);
        }
        insertTerm(p, coeff, exp);
    }
}
/* Display polynomial in readable mathematical form */
void displayPolynomial(const Polynomial *p) {
    if (p->tail == NULL) {
        printf("0");
        return;
    }
    Node *head = p->tail->next;
    Node *cur = head;
    int first = 1;
    do {
        int c = cur->coeff;
        int absCoeff = (c < 0) ? -c : c;
        if (!first) {
            printf(c < 0 ? " - " : " + ");
        } else if (c < 0) {
            printf("-");
        }
        if (cur->exp == 0) {
            printf("%d", absCoeff);
        } else {
            if (absCoeff != 1) printf("%d", absCoeff);
            printf("x");
            if (cur->exp != 1) printf("^%d", cur->exp);
        }
        first = 0;
        cur = cur->next;
    } while (cur != head);
}
/* Add two polynomials into a new circular linked list */
Polynomial addPolynomials(const Polynomial *a, const Polynomial *b) {
    Polynomial result;
    initPolynomial(&result);
    if (a->tail != NULL) {
        Node *head = a->tail->next, *cur = head;
        do {
            insertTerm(&result, cur->coeff, cur->exp);
            cur = cur->next;
        } while (cur != head);
    }
    if (b->tail != NULL) {
        Node *head = b->tail->next, *cur = head;
        do {
            insertTerm(&result, cur->coeff, cur->exp);
            cur = cur->next;
        } while (cur != head);
    }
    return result;
}
/* Free all nodes in a circular list */
void freePolynomial(Polynomial *p) {
    if (p->tail == NULL) return;
    Node *head = p->tail->next;
    p->tail->next = NULL;
    Node *cur = head;
    while (cur != NULL) {
        Node *next = cur->next;
        free(cur);
        cur = next;
    }
    p->tail = NULL;
}
int main(void) {
    Polynomial p1, p2, sum;
    initPolynomial(&p1);
    initPolynomial(&p2);
    initPolynomial(&sum);
    printf("=== Polynomial Addition using Circular Linked List ===\\n");
    printf("\\nCreate Polynomial 1\\n");
    createPolynomial(&p1);
    printf("P1(x) = ");
    displayPolynomial(&p1);
    printf("\\n");
    printf("\\nCreate Polynomial 2\\n");
    createPolynomial(&p2);
    printf("P2(x) = ");
    displayPolynomial(&p2);
    printf("\\n");
    sum = addPolynomials(&p1, &p2);
```

```
    printf("\\nResultant Polynomial\\nP1(x) + P2(x) = ");
    displayPolynomial(&sum);
    printf("\\n");
    freePolynomial(&p1);
    freePolynomial(&p2);
    freePolynomial(&sum);
    return 0;
}
```

Sample Input and Output (Terminal Screenshot)

```
$ gcc -Wall -Wextra -std=c11 polynomial_circular.c -o polynomial
$ ./polynomial
=== Polynomial Addition using Circular Linked List ===

Create Polynomial 1
Enter number of terms: 3
Enter coefficient and exponent for term 1: 5 3
Enter coefficient and exponent for term 2: 2 2
Enter coefficient and exponent for term 3: -4 1
P1(x) = 5x^3 + 2x^2 - 4x

Create Polynomial 2
Enter number of terms: 3
Enter coefficient and exponent for term 1: 3 3
Enter coefficient and exponent for term 2: 7 1
Enter coefficient and exponent for term 3: 6 0
P2(x) = 3x^3 + 7x + 6

Resultant Polynomial
P1(x) + P2(x) = 8x^3 + 2x^2 + 3x + 6
$
```

Sample used: P1(x) = $5x^3 + 2x^2 – 4x$ and P2(x) = $3x^3 + 7x + 6$. The resultant polynomial is $8x^3 + 2x^2 + 3x + 6$.